

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python.

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data.

Arrays in Numpy

Array in Numpy is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers. In Numpy, number of dimensions of the array is called rank of the array. A tuple of integers giving the size of the array along each dimension is known as shape of the array. An array class in Numpy is called as **ndarray**. Elements in Numpy arrays are accessed by using square brackets and can be initialized by using nested Python Lists.

Using NumPy (Recommended)

First, ensure you have NumPy installed (`pip install numpy`). Then, import the library and use its built-in functionalities. [🔗](#)

Creating a Numpy Array

Arrays in Numpy can be created by multiple ways, with various number of Ranks, defining the size of the Array. Arrays can also be created with the use of various data types such as lists, tuples, etc. The type of the resultant array is deduced from the type of the elements in the sequences.

Note: Type of array can be explicitly defined while creating the array.

```
import numpy as np

# Creating a rank 1 Array
arr = np.array([1, 2, 3])
print(arr)

# Creating a rank 2 Array
arr = np.array([[1, 2, 3],
                [4, 5, 6]])
print(arr)

# Creating an array from tuple
arr = np.array((1, 3, 2))
print(arr)
```

Output

```
Array with Rank 1:  
[1 2 3]  
Array with Rank 2:  
[[1 2 3]  
 [4 5 6]]  
  
Array created using passed tuple:  
[1 3 2]
```

Accessing Single Elements

Indexing in [NumPy](#) is zero-based, meaning the first element is at index

0. [🔗](#)

Accessing the array Index

In a numpy array, indexing or accessing the array index can be done in multiple ways. To print a range of an array, slicing is done. Slicing of an array is defining a range in a new array which is used to print a range of elements from the original array. Since, sliced array holds a range of elements of the original array, modifying content with the help of sliced array modifies the original array content.

```
import numpy as np  
  
arr = np.array([[-1, 2, 0, 4],  
                [4, -0.5, 6, 0],  
                [2.6, 0, 7, 8],  
                [3, -7, 4, 2.0]])
```

- **1D Array:** Use a single integer index.

python



```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[0]) # Output: 10
print(arr[4]) # Output: 50
```

- **2D Array:** Use comma-separated indices for the row and column:

```
arr[row_index, column_index].
```

python



```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[1, 2]) # Output: 6 (second row, third column)
print(matrix[0, 0]) # Output: 1 (first row, first column)
```

- **Negative Indexing:** Access elements from the end of the array using negative indices (e.g., `-1` for the last element).

python



```
print(arr[-1]) # Output: 50
print(matrix[-1, -1]) # Output: 9
```

Slicing extracts a range of elements using the syntax

[start:stop:step]. The stop index is excluded from the result. [🔗](#)

- **1D Array Slicing:**

python



```
print(arr[1:4])    # Output: [20 30 40] (from index 1 up to, but
print(arr[2:])     # Output: [30 40 50] (from index 2 to the end)
print(arr[:3])     # Output: [10 20 30] (from the beginning up to
print(arr[::-2])   # Output: [10 30 50] (every other element)
```

- **2D Array Slicing:** Apply slicing to each dimension separately using commas.

python



```
# Access the entire first row
print(matrix[0, :]) # or simply matrix[0]
# Output: [1 2 3]

# Access the second column (all rows, index 1 column)
print(matrix[:, 1])
# Output: [2 5 8]

# Access a sub-array (first two rows, first two columns)
print(matrix[:2, :2])
# Output:
# [[1 2]
#  [4 5]]
```

Extract a Row

To extract an entire row, specify the row's index and use a colon `:` for the column index. The colon indicates that you want all columns. [🔗](#)

- **Syntax:** `arr[row_index, :]` or simply `arr[row_index]`
- The result will be a 1D array by default. [🔗](#)

Action 🔗	Code	Result
Extract the first row (index 0)	<code>arr[0, :]</code> or <code>arr[0]</code>	<code>array([1, 2, 3])</code>
Extract the second row (index 1)	<code>arr[1, :]</code> or <code>arr[1]</code>	<code>array([4, 5, 6])</code>
Extract the last row	<code>arr[-1, :]</code> or <code>arr[-1]</code>	<code>array([7, 8, 9])</code>

To return the row as a 2D array (useful for maintaining dimensions for some operations), use a slice for the row index as well: `arr[0:1, :]`. [🔗](#)

Extract a Column

To extract an entire column, use a colon `:` for the row index and specify the column's index. [🔗](#)

- **Syntax:** `arr[:, column_index]`
- The result will be a 1D array by default. [🔗](#)

Action 🔗	Code	Result
Extract the first column (index 0)	<code>arr[:, 0]</code>	<code>array([1, 4, 7])</code>
Extract the second column (index 1)	<code>arr[:, 1]</code>	<code>array([2, 5, 8])</code>
Extract the last column	<code>arr[:, -1]</code>	<code>array([3, 6, 9])</code>

To return the column as a 2D array, use a slice for the column index: `arr[:, 0:1]`. [🔗](#)

Create Matrix in NumPy

In NumPy, we use the `np.array()` function to create a matrix. For example,

```
import numpy as np

# create a 2x2 matrix
matrix1 = np.array([[1, 3],
                    [5, 7]])

print("2x2 Matrix:\n",matrix1)
```



Output

```
2x2 Matrix:
[[1 3]
 [5 7]]
```

1. Matrix Sum (Addition)

For two matrices to be compatible for addition, they **must have the same dimensions**. NumPy's `+` operator handles this automatically. [🔗](#)

python



```
import numpy as np

# Define two compatible matrices (e.g., 2x2)
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Calculate the sum
C = A + B
# or using np.add()
# C = np.add(A, B)

print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
print("\nSum of A and B:")
print(C)
```

Output:

```
Matrix A:
[[1 2]
 [3 4]]

Matrix B:
[[5 6]
 [7 8]]

Sum of A and B:
[[ 6  8]
 [10 12]]
```

Perform Matrix Multiplication in NumPy

We use the `np.dot()` function to perform multiplication between two matrices.

Let's see an example.

```
import numpy as np

# create two matrices
matrix1 = np.array([[1, 3],
                    [5, 7]])

matrix2 = np.array([[2, 6],
                    [4, 8]])

# calculate the dot product of the two matrices
result = np.dot(matrix1, matrix2)

print("matrix1 x matrix2: \n",result)
```

Run Code »

Output

```
matrix1 x matrix2:
[[14 30]
 [38 86]]
```


Transpose NumPy Matrix

The transpose of a matrix is a new matrix that is obtained by exchanging the rows and columns. For 2x2 matrix,

```
Matrix:
a11  a12
a21  a22

Transposed Matrix:
a11  a21
a12  a22
```

In NumPy, we can obtain the transpose of a matrix using the `np.transpose()` function. For example,

```
import numpy as np

# create a matrix
matrix1 = np.array([[1, 3],
                    [5, 7]])

# get transpose of matrix1
result = np.transpose(matrix1)

print(result)
```

Calculate Inverse of a Matrix in NumPy

In NumPy, we use the `np.linalg.inv()` function to calculate the inverse of the given matrix.

However, it is important to note that not all matrices have an inverse. Only square matrices that have a non-zero determinant have an inverse.

Now, let's use `np.linalg.inv()` to calculate the inverse of a square matrix.

```
import numpy as np

# create a 3x3 square matrix
matrix1 = np.array([[1, 3, 5],
                    [7, 9, 2],
                    [4, 6, 8]])

# find inverse of matrix1
result = np.linalg.inv(matrix1)

print(result)
```

Run Code »

Output

```
[[-1.11111111 -0.11111111  0.72222222]
 [ 0.88888889  0.22222222 -0.61111111]
 [-0.11111111 -0.11111111  0.22222222]]
```

Note: If we try to find the inverse of a non-square matrix, we will get an error

message: `numpy.linalg.linalgerror: Last 2 dimensions of the array must be square`

Find Determinant of a Matrix in NumPy

We can find the determinant of a square matrix using the `np.linalg.det()` function to calculate the determinant of the given matrix.

Suppose we have a **2x2** matrix A :

```
a b  
c d
```

So, the determinant of a **2x2** matrix will be:

```
det(A) = ad - bc
```

where **a**, **b**, **c**, and **d** are the elements of the matrix.

Let's see an example.

```
import numpy as np  
  
# create a matrix  
matrix1 = np.array([[1, 2, 3],  
                    [4, 5, 1],  
                    [2, 3, 4]])  
  
# find determinant of matrix1  
result = np.linalg.det(matrix1)  
  
print(result)
```



Run Code >>

Output

-5.00

Flatten Matrix in NumPy

Flattening a matrix simply means converting a matrix into a 1D array.

To flatten a matrix into a 1-D array we use the `array.flatten()` function. Let's see an example.

```
import numpy as np

# create a 2x3 matrix
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 7]])

result = matrix1.flatten()
print("Flattened 2x3 matrix:", result)
```

Run Code »

Output

```
Flattened 2x3 matrix: [1 2 3 4 5 7]
```

Here, we have used the `matrix1.flatten()` function to flatten `matrix1` into a 1D array, without compromising any of its elements

Method 1: Using NumPy for Linear Equations (Numerical Solution)

The most common method for solving a system of linear equations of the form $Ax = b$ is using the `numpy.linalg.solve()` function. This method is fast and numerically stable. [🔗](#)

1. Using NumPy for Linear Equations

NumPy is ideal for systems of linear equations with numerical coefficients. The system must be represented in the matrix form $A\mathbf{x} = \mathbf{b}$, where A is the coefficient matrix and $\mathbf{b}$ is the vector of constants. [🔗](#)

Consider the system of equations:

- $2x + y = 5$
- $x + 3y = 7$

The code to solve this using `numpy.linalg.solve()` is:

python

```
import numpy as np

# Define the coefficient matrix A
A = np.array([[2, 1],
              [1, 3]])

# Define the constants vector b
b = np.array([5, 7])

# Solve for the variables x and y
solution = np.linalg.solve(A, b)

print("Solution:", solution)
print("x =", round(solution[0]))
print("y =", round(solution[1]))
```

Output:

```
Solution: [2. 1.]
x = 2
y = 1
```